

GEMINI ENTERPRISE AGENT PLATFORM

Evaluating Agents on the Agent Platform

The Quality Flywheel. Measure, monitor, and keep improving agent quality end to end. A hands-on walkthrough that covers everything in the Optimize / Evaluation docs, run against real deployed agents.

Design > Execution > Scoring > Refinement

```
$ uv run python -m src.eval.demo.full_eval_demo --agent-id $AGENT_ENGINE_ID
```

[Docs: Optimize / Evaluation ↗](#) · [github.com/jswoertz/geap-tour ↗](https://github.com/jswoertz/geap-tour) · [Demo walkthrough ↗](#)

THE QUALITY FLYWHEEL

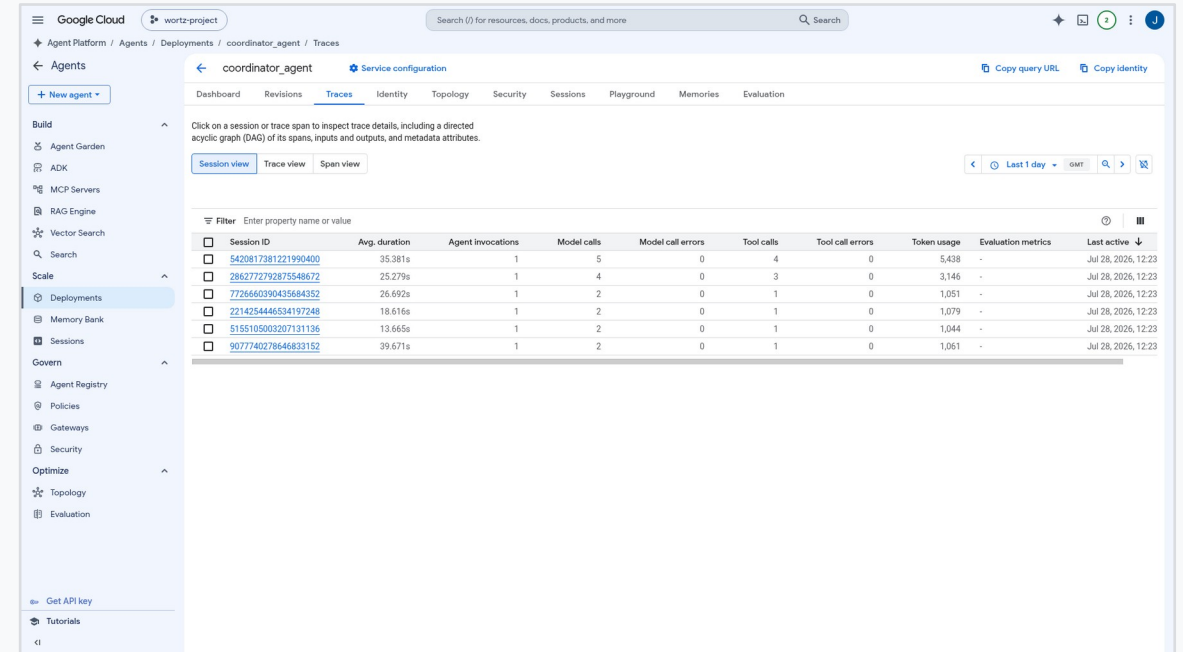
Evaluating agents on the Agent Platform

Turn “it seems to work” into something you can measure. Evaluation is a loop across four phases: Design, Execution, Scoring, Refinement.

- **Define** eval cases and metrics, **run** inferences to produce traces, **score** them with autoraters, then **analyze** and **optimize**. Repeat.
- Covers **all 9 of the “Optimize / Evaluation” doc pages**, running against real deployed agents.
- Run it: `uv run python -m src.eval.demo.full_eval_demo --agent-id $AGENT_ENGINE_ID`

[Docs: Agent evaluation overview ↗](#)

[docs/evaluation_demo.md](#) [docs/eval_operations.md](#)



Session ID	Avg. duration	Agent invocations	Model calls	Model call errors	Tool calls	Tool call errors	Token usage	Evaluation metrics	Last active
5420817381221990400	35.381s	1	5	0	4	0	5,438	-	Jul 28, 2025, 12:23
2862772792875548672	25.279s	1	4	0	3	0	3,146	-	Jul 28, 2025, 12:23
7726660390435684352	26.692s	1	2	0	1	0	1,051	-	Jul 28, 2025, 12:23
2214254446534197248	18.616s	1	2	0	1	0	1,079	-	Jul 28, 2025, 12:23
5155105905207131136	13.666s	1	2	0	1	0	1,044	-	Jul 28, 2025, 12:23
9077740278646833152	39.671s	1	2	0	1	0	1,061	-	Jul 28, 2025, 12:23

DEFINE “GOOD” · METRIC REGISTRY

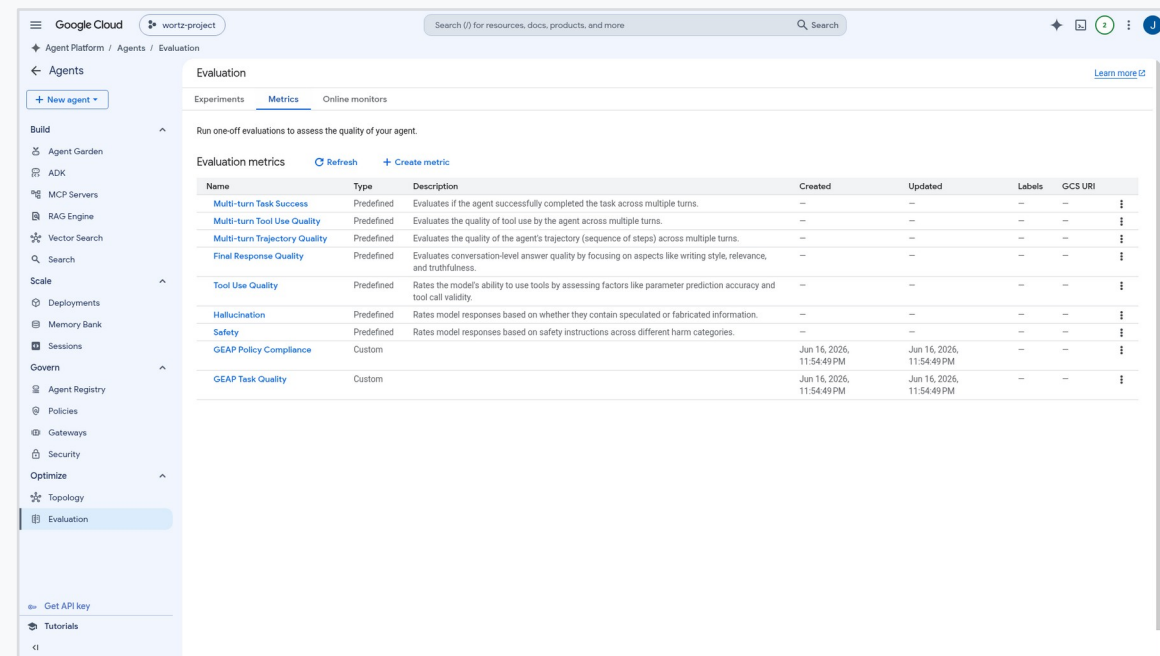
Pick a metric, or **write** your own

Define a metric once and reuse it across offline runs and online monitors. There are three kinds, plus the split between reference-based and reference-free.

- **Predefined rubric**: single-turn (Final Response Quality, Hallucination, Tool Use, Safety) and multi-turn (Task Success, Tool Use, Trajectory).
- **Adaptive** rubrics write their own criteria; **static** ones score 0 to 1.
- **Custom LLM-as-judge** (types.LLMMetric) and **custom code** (types.CodeExecutionMetric).
- **Reference-based** Exact Match vs **reference-free** rubrics.

[Docs: Manage evaluation metrics ↗](#)

```
</> src/eval/metric_registry.py </> src/eval/agent_eval_configs.py
```



The screenshot shows the Google Cloud console interface for the 'Evaluation' section. The left sidebar contains a navigation menu with categories like Build, Scale, Govern, and Optimize. The main content area displays a table of 'Evaluation metrics' with columns for Name, Type, Description, Created, Updated, Labels, and GCS URI. The table lists several predefined metrics such as 'Multi-turn Task Success', 'Multi-turn Tool Use Quality', 'Multi-turn Trajectory Quality', 'Final Response Quality', 'Tool Use Quality', 'Hallucination', 'Safety', and 'GEAP Policy Compliance'. It also shows two custom metrics: 'GEAP Task Quality' and 'GEAP Policy Compliance'.

Name	Type	Description	Created	Updated	Labels	GCS URI
Multi-turn Task Success	Predefined	Evaluates if the agent successfully completed the task across multiple turns.	—	—	—	—
Multi-turn Tool Use Quality	Predefined	Evaluates the quality of tool use by the agent across multiple turns.	—	—	—	—
Multi-turn Trajectory Quality	Predefined	Evaluates the quality of the agent's trajectory (sequence of steps) across multiple turns.	—	—	—	—
Final Response Quality	Predefined	Evaluates conversation-level answer quality by focusing on aspects like writing style, relevance, and truthfulness.	—	—	—	—
Tool Use Quality	Predefined	Rates the model's ability to use tools by assessing factors like parameter prediction accuracy and tool call validity.	—	—	—	—
Hallucination	Predefined	Rates model responses based on whether they contain speculated or fabricated information.	—	—	—	—
Safety	Predefined	Rates model responses based on safety instructions across different harm categories.	—	—	—	—
GEAP Policy Compliance	Custom		Jun 16, 2026, 11:54:49 PM	Jun 16, 2026, 11:54:49 PM	—	—
GEAP Task Quality	Custom		Jun 16, 2026, 11:54:49 PM	Jun 16, 2026, 11:54:49 PM	—	—

GENERATE TRACES

Three ways to exercise the agent

A trace is the record of one run: the model inputs, the responses, and the tool calls. Produce them whichever way fits the stage.

- **Batch / test-case**: run inference over a fixed suite for regression testing in CI.
- **Simulated**: generate multi-turn scenarios and let an LLM play the user, grounded by environment_context. Inject mocked tools or 503s.
- **Offline**: score **already-recorded** historical traces and sessions in Cloud Trace and BigQuery, with **no new inference**.

[Docs: Run evaluation / Simulate / Offline ↗](#)

`</> multi_agent_batch_eval.py` `</> simulated_eval.py` `</> env_simulation.py` `</> offline_trace_eval.py`

Name	Resource name	Identity	Telemetry collection	Created	Framework	Region
party-store-ge-a2ui	../4796351195335622656	...gserviceaccount.com	Learn more	Jul 9, 2026, 2:46:15 AM	custom (a2a)	us-east1 (South Carolina)
router_agent	../2985691389632708608	../2985691389632708608	Enabled Configure	Jun 30, 2026, 4:46:19 PM	google-adk	us-central1 (Iowa)
coordinator_agent	../5895016748914049024	../5895016748914049024	Enabled Configure	Jun 30, 2026, 4:41:12 PM	google-adk	us-central1 (Iowa)
claude-ge-agent	../7508732031849725952	...gserviceaccount.com	Enabled Configure	Jun 19, 2026, 12:48:21 AM	google-adk	us-east1 (South Carolina)
Test Low Code Agent	../7935421708264013824	...gserviceaccount.com	Enabled Configure	May 26, 2026, 9:25:12 PM	google-adk	us-west1 (Oregon)
AGENT_DESIGNER_GENERATED_DO_NOT_DELETE	../2319406584653938688	...gserviceaccount.com	Learn more	Feb 26, 2026, 12:01:06 AM	google-adk	us-west1 (Oregon)

CONTINUOUS EVAL & GUARDRAILS

Watch quality in production

Online Monitors score live traffic as it comes in and send the scores to Cloud Monitoring, so you catch quality drift before users do.

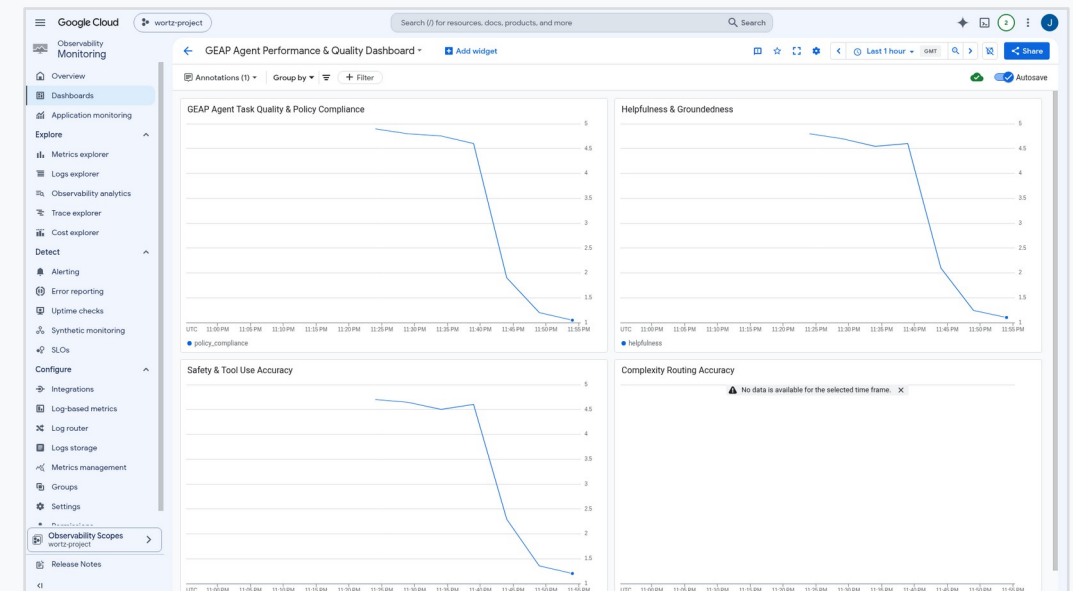
- **Online Monitors:** a roughly 10-minute loop that queries traffic, evaluates it, and reports. Sampling is configurable, and results land in Logging and Monitoring.
- **Metrics Explorer and dashboards:** every score becomes a Cloud Monitoring metric (agent_eval/*, online_evaluator/scores), so you can chart, compare, and share it.
- **Quality-drift alerts:** a per-monitor policy, the dashboard's Recommended Alerts, or gcloud / monitoring_v3.

[Docs: Online monitors / Quality alerts ↗](#)

```
</> setup_online_evaluators.py </> publish_metrics.py </> q
uality_alerts.py
```

The screenshot shows the 'Evaluation' section of the Google Cloud Agent Platform. It features a table of 'Online monitors' with columns for Monitor ID, Agent, Status, Filter, Sampling, Region, Date created, and Sampled traces. All monitors are listed as 'Active'.

Monitor	Agent	Status	Filter	Sampling	Region	Date created	Sampled traces
Monitor_6514879073456488448	3003309963556131232	Active		100%	us-central1	Jun 17, 2026, 12:34:30 AM	View traces
Monitor_8365858520305762304	1240563796693824	Active		100%	us-central1	Jun 17, 2026, 3:06:18 PM	View traces
Monitor_8424252548926144512	coordinator_agent	Active		100%	us-central1	Jun 30, 2026, 5:48:01 PM	View traces
Monitor_544921481629944832	router_agent	Active		100%	us-central1	Jun 30, 2026, 5:48:01 PM	View traces



ANALYZE, OPTIMIZE, REPEAT

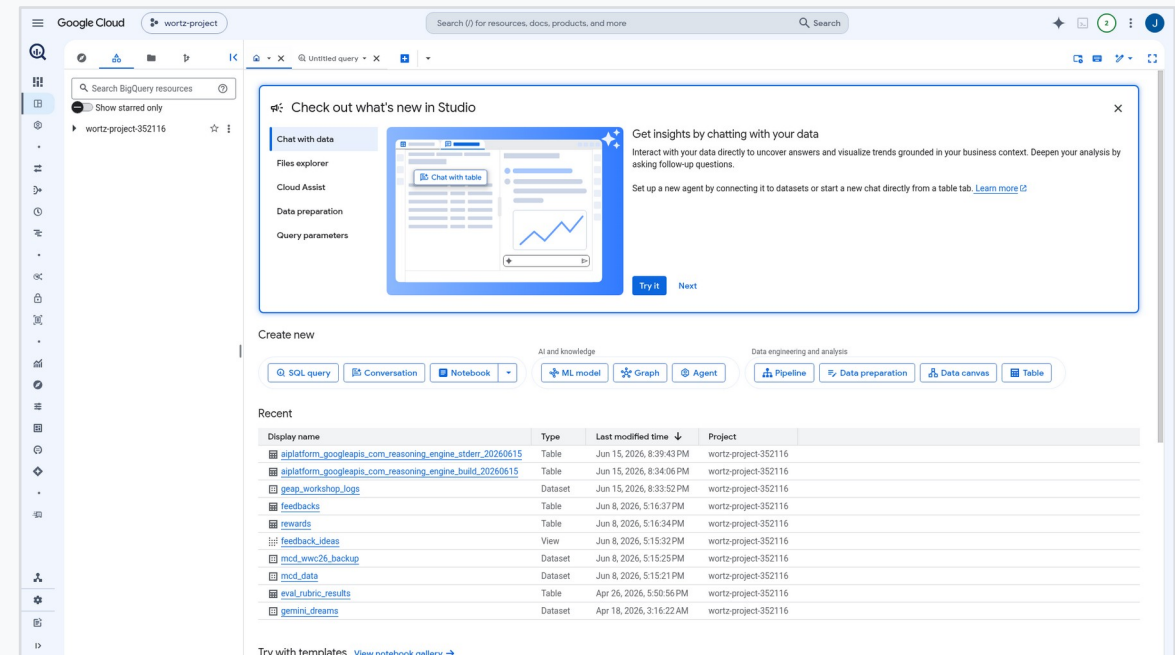
Close the flywheel

Don't just learn that the agent failed. Learn why, then fix it.

- **Optimize:** refine root instructions with **GEPA** or **SimplePromptOptimizer**, or drive the loop from an assistant with agents-cli eval.
- **Re-evaluate:** score the tuned agent against the same suite and compare, closing the loop.
- **Built clean:** current Google clients from PyPI, with 161 passing tests.

[Docs: Optimize agent prompts ↗](#)

```
</> sdk_optimize.py </> run_optimize.py </> agents_cli_demo.sh
```



THE QUALITY FLYWHEEL IN ONE VIEW

From “seems to work” to measured & monitored

- **Design:** a reusable Metric Registry with predefined rubrics, custom LLM-as-judge, custom code, and reference-based Exact Match.
- **Execution:** generate traces three ways, batch in CI, simulated (with user and environment simulation), and offline over historical traces.
- **Scoring:** Online Monitors export every score as a Cloud Monitoring metric, visible in Metrics Explorer, dashboards, and drift alerts.
- **Refinement:** optimize the agent's instructions with GEPA, SimplePromptOptimizer, or agents-cli, then re-evaluate. Repeat.

Built clean: current Google clients from PyPI, with 161 passing tests.

```
$ uv run python -m src.eval.demo.full_eval_demo --agent-id $AGENT_ENGINE_ID
```

[Docs: Optimize / Evaluation ↗](#)

[Coverage matrix \(\\$0\) ↗](#)

[Demo walkthrough ↗](#)